Anexo — Pruebas y cobertura del orquestador

Fortia (Micro-SOAR) · Plataforma móvil de respuesta a incidentes · Proyecto Final de Ingeniería

> Generado el 18 de octubre de 2026 a partir de la corrida real de la suite sobre el commit `a526454`. Entorno: Node.js v24.19.0, Jest 29.7.0, base SQLite efímera. Regenerable con `docs/anexo-pruebas/generar.js`.

1. Resumen

210/210	21	98,9%	92,4%
tests aprobados	archivos de prueba	cobertura de líneas	cobertura de ramas

El orquestador (backend Node.js/Express que recibe las alertas de Wazuh y los reportes de phishing, los normaliza en incidentes, los enriquece y ejecuta las acciones de respuesta) supera el umbral de **85%** exigido en las cuatro métricas de cobertura. El umbral no es solo un objetivo: está configurado en `coverageThreshold` y el pipeline de integración continua rechaza cualquier cambio que lo haga descender.

Las pruebas automatizadas se complementan con pruebas de punta a punta contra la infraestructura real en AWS (sección 5), que validan lo que los mocks no pueden: el comportamiento efectivo de Wazuh, del firewall del host y de la app en un dispositivo físico.

2. Estrategia de pruebas

Nivel	Qué valida	Cómo
Unitario	Normalización y filtrado de alertas; cliente de la API de Wazuh; enriquecimiento	Jest. Las llamadas HTTP externas (Wazuh, VirusTotal, AbuseIPDB) se reemplazan por mocks de `axios` que reproducen respuestas reales, incluidos los errores; el servidor de correo, por un cliente IMAP simulado con buzones en memoria.
Integración (API)	Endpoints REST completos: webhook, incidentes, acciones de respuesta, autenticación, auditoría	Supertest contra la aplicación Express real y una base SQLite efímera creada con Prisma en cada corrida. Solo se aísla la frontera con Wazuh.
Punta a punta	Cadena completa ataque → detección → incidente → app → acción → efecto en la red	Manual y documentada, contra la infraestructura de AWS y un celular real por Tailscale (sección 5).

Criterios de diseño

- **Probar los modos de falla, no solo el camino feliz.** La mayoría de los defectos del proyecto fueron respuestas engañosas (Wazuh aceptando una acción que luego no ejecuta), por lo que cada acción de respuesta se prueba contra rechazo HTTP, rechazo con HTTP 200 y agente fallido.
- **Los servicios auxiliares no deben interrumpir el flujo crítico.** Se verifica que una caída de VirusTotal o de la auditoría no impida registrar un incidente ni iniciar sesión.
- **Sin dependencia del entorno local.** Las pruebas corren sin credenciales ni archivo `.env`; ninguna llega a la red. Así se ejecutan idénticas en la notebook de desarrollo y en GitHub Actions.
- **Datos de prueba realistas.** Las alertas simuladas (`orchestrator/mocks/`) replican alertas reales de Wazuh: fuerza bruta SSH (regla 5712), malware detectado por VirusTotal (87105) y ruido de integridad de archivos

(550).

3. Cobertura de código

Métrica	Cubierto	Total	Cobertura	Umbral	Estado
Sentencias	1207	1228	98,3%	85%	**Cumple**
Ramas (branches)	756	818	92,4%	85%	**Cumple**
Funciones	233	243	95,9%	85%	**Cumple**
Líneas	1023	1034	98,9%	85%	**Cumple**

Por archivo

Archivo	Sentencias	Ramas	Funciones	Líneas
`src/app.js`	96,9%	91,2%	90,0%	97,8%
`src/audit.js`	93,8%	71,4%	100,0%	100,0%
`src/auth.js`	100,0%	95,3%	100,0%	100,0%
`src/automation.js`	98,5%	98,0%	100,0%	98,3%
`src/campaign.js`	97,2%	91,2%	100,0%	98,8%
`src/closure.js`	100,0%	92,8%	100,0%	100,0%
`src/db.js`	100,0%	100,0%	100,0%	100,0%
`src/email.js`	98,8%	94,2%	93,3%	100,0%
`src/emailActions.js`	100,0%	90,0%	100,0%	100,0%
`src/emailParse.js`	100,0%	74,1%	100,0%	100,0%
`src/enrichment.js`	98,2%	100,0%	91,7%	97,9%
`src/grouping.js`	100,0%	88,9%	100,0%	100,0%
`src/mailProvider.js`	98,9%	96,0%	87,0%	100,0%
`src/metrics.js`	100,0%	100,0%	100,0%	100,0%
`src/mitigation.js`	100,0%	83,3%	100,0%	100,0%
`src/normalize.js`	100,0%	93,9%	100,0%	100,0%
`src/privacy.js`	100,0%	91,2%	100,0%	100,0%
`src/wazuh.js`	100,0%	100,0%	100,0%	100,0%
Total	**98,3%**	**92,4%**	**95,9%**	**98,9%**

La medición abarca todo el código de la aplicación (`orchestrator/src/`). Quedan fuera los scripts operativos de uso único (`block_ip.js`, prueba de concepto de la fase 0, y `scripts/limpiar-ruido.js`). Las únicas líneas no ejecutadas corresponden al arranque del servidor HTTP (`app.listen`): las pruebas montan la aplicación en memoria sin abrir un puerto.

Aplicación del umbral en integración continua

Cada *pull request* hacia la rama principal ejecuta la suite completa en GitHub Actions. Si la cobertura de cualquiera de las cuatro métricas cae por debajo del 85%, el job falla y el cambio no debe integrarse. El resultado se publica como tabla en el resumen de la ejecución y el reporte HTML navegable (cobertura línea por línea) queda disponible como artefacto descargable. Se verificó que el mecanismo funciona forzando un umbral de 99,5%: la suite falló con `coverage threshold for lines (99.5%) not met`.

4. Catálogo de pruebas automatizadas

Cada grupo indica la parte del sistema que valida, el caso de uso al que pertenece y el riesgo que mitiga.

Ingesta y normalización · CU-01 · `webhook.test.js`

Riesgo mitigado: Una alerta de Wazuh mal formada o incompleta corrompe la base o tira el servicio.

Caso de prueba	Resultado
el webhook normaliza y persiste una alerta de brute-force SSH	**Aprobado**
el webhook rechaza una alerta con campos faltantes	**Aprobado**
GET /incidents devuelve la alerta persistida	**Aprobado**

Ingesta de malware · CU-02 · `webhookMalware.test.js`

Riesgo mitigado: Una alerta sin IP atacante (FIM/VirusTotal) se rechaza y el incidente se pierde. Defecto real encontrado el 02/10.

Caso de prueba	Resultado
el webhook acepta una alerta de VirusTotal/FIM (CU-02) sin srcIp	**Aprobado**
block-ip rechaza con 422 un incidente sin srcIp (ej. CU-02)	**Aprobado**

Filtro de ruido · CU-01 / CU-02 · `alertFilter.test.js`

Riesgo mitigado: Fatiga de alertas: 1.320 de 1.334 incidentes eran ruido (13/10).

Caso de prueba	Resultado
una alerta de FIM sobre archivos del sistema no crea incidente	**Aprobado**
una alerta ignorada igual se valida: si esta rota da 422	**Aprobado**
solo pasan las familias de los casos de uso	**Aprobado**

Respuesta: bloquear IP · CU-01 · `blockIp.test.js`

Riesgo mitigado: El incidente queda marcado como mitigado aunque Wazuh haya rechazado la acción.

Caso de prueba	Resultado
bloquea un incidente existente y lo marca como blocked	**Aprobado**
devuelve 404 si el incidente no existe	**Aprobado**
devuelve 502 si Wazuh rechaza el comando	**Aprobado**

Respuesta: aislar host · CU-02 · `isolateHost.test.js`

Riesgo mitigado: Idem, para el aislamiento del endpoint.

Caso de prueba	Resultado
aisla un incidente existente y lo marca como isolated	**Aprobado**
devuelve 404 si el incidente no existe	**Aprobado**
devuelve 502 si Wazuh rechaza el comando	**Aprobado**

Cliente de la API de Wazuh · CU-01 / CU-02 · `wazuh.test.js`

Riesgo mitigado: Wazuh responde HTTP 200 con la acción fallida en el cuerpo (error 1703, 04/08): la app informaría un bloqueo que no ocurrió.

Caso de prueba	Resultado
authenticate — devuelve el token JWT del manager	**Aprobado**
authenticate — credenciales rechazadas -> error con el status	**Aprobado**
blockIp — HTTP 200 sin items fallidos -> ok	**Aprobado**
blockIp — status HTTP de error -> rechaza	**Aprobado**
blockIp — HTTP 200 pero error en el body -> rechaza	**Aprobado**
blockIp — HTTP 200 pero el agente fallo (ej. 1703 en el agente 000) -> rechaza	**Aprobado**
isolateHost — HTTP 200 sin items fallidos -> ok	**Aprobado**
isolateHost — status HTTP de error -> rechaza	**Aprobado**
isolateHost — HTTP 200 pero error en el body -> rechaza	**Aprobado**
isolateHost — HTTP 200 pero el agente fallo (ej. 1703 en el agente 000) -> rechaza	**Aprobado**
ninguna llamada deja que axios tire por status HTTP: el status se evalua a mano	**Aprobado**
blockIp manda la IP en alert.data.srcip (lo que lee firewall-drop)	**Aprobado**
isolateHost manda !isolate-host sin IP (aisla al propio agente)	**Aprobado**

Enriquecimiento (VirusTotal / AbuseIPDB) · CU-01 · `enrichment.test.js`

Riesgo mitigado: Cuota gratuita agotada, red caída o API caída bloquean o rompen la ingesta.

Caso de prueba	Resultado
sin API keys configuradas, no pega a la red y devuelve null en ambas	**Aprobado**
con ambas API keys, consulta VT y AbuseIPDB y resume la respuesta	**Aprobado**
cachea el resultado: una segunda consulta a la misma IP no vuelve a pegarle a la red	**Aprobado**
si la red falla (timeout, DNS), no rompe: devuelve null para esa fuente	**Aprobado**
con una sola API key, consulta solo esa fuente	**Aprobado**
si la API responde con error, no rompe: devuelve null para esa fuente	**Aprobado**
reputacion de URLs y archivos (CU-03) — sin API key de VT no consulta	**Aprobado**
reputacion de URLs y archivos (CU-03) — consulta la URL por su base64url y el archivo por su hash, con cache	**Aprobado**
reputacion de URLs y archivos (CU-03) — objeto desconocido para VT (404) o error de red: null	**Aprobado**

Consulta y robustez · Transversal · `incidents.test.js`

Riesgo mitigado: Fallas de servicios auxiliares (enriquecimiento, auditoría) interrumpen el flujo principal.

Caso de prueba	Resultado
GET /incidents/:id devuelve el incidente	**Aprobado**
GET /incidents/:id inexistente -> 404	**Aprobado**
un token adulterado o vencido -> 401	**Aprobado**
el webhook guarda el enriquecimiento cuando VT/AbuseIPDB responden	**Aprobado**
sin datos de enriquecimiento, el incidente queda sin tocar	**Aprobado**
si el enriquecimiento falla, el webhook igual responde 201	**Aprobado**
si falla la auditoria, el login igual funciona (la auditoria no es bloqueante)	**Aprobado**

Autenticación (JWT) · Seguridad · `auth.test.js`

Riesgo mitigado: Acceso a incidentes o acciones sin credenciales válidas.

Caso de prueba	Resultado
login devuelve un token con credenciales validas	**Aprobado**
login rechaza credenciales invalidas	**Aprobado**
GET /incidents sin token devuelve 401	**Aprobado**

Perfil del analista · Soporte · `me.test.js`

Riesgo mitigado: Datos de perfil inválidos o accesibles sin autenticación.

Caso de prueba	Resultado
GET /me sin token devuelve 401	**Aprobado**
GET /me crea y devuelve el perfil del usuario autenticado	**Aprobado**
PUT /me actualiza nombre y email	**Aprobado**
PUT /me sin name/email devuelve 422	**Aprobado**

Auditoría · Seguridad · `audit.test.js`

Riesgo mitigado: Acciones del analista sin registro trazable.

Caso de prueba	Resultado
GET /audit sin token devuelve 401	**Aprobado**
un login exitoso queda registrado en el audit log	**Aprobado**

Usuarios y control de acceso por roles · Seguridad (RF-13) · `users.test.js`

Riesgo mitigado: Acceso sin aprobación, un rol que excede sus permisos, sesiones que siguen vivas después de desactivar una cuenta, o el sistema sin ningún administrador.

Caso de prueba	Resultado
politica de contrasenas — exige 9+ caracteres con mayuscula, minuscula, numero y especial	**Aprobado**
registro con aprobacion — crea la cuenta pendiente, como analista y con la contrasena hasheada	**Aprobado**
registro con aprobacion — una cuenta pendiente no puede entrar	**Aprobado**
registro con aprobacion — un admin la aprueba y recien ahi puede entrar	**Aprobado**
registro con aprobacion — rechaza datos incompletos, contrasena debil y usuario repetido	**Aprobado**
login — misma respuesta para usuario inexistente y contrasena incorrecta	**Aprobado**
login — una cuenta desactivada no puede entrar	**Aprobado**
permisos por rol — el auditor ve incidentes, auditoria y metricas, pero no puede mitigar	**Aprobado**
permisos por rol — el analista mitiga, pero no gestiona usuarios	**Aprobado**
permisos por rol — el admin lista usuarios (pendientes primero, sin hashes)	**Aprobado**
permisos por rol — el rol se toma de la base: un cambio de rol aplica sin volver a loguearse	**Aprobado**
permisos por rol — desactivar a un usuario corta su sesion en el acto	**Aprobado**
gestion de usuarios (admin) — valida rol y estado, y responde 404 si el usuario no existe	**Aprobado**
gestion de usuarios (admin) — audita cambios de estado y de rol, y un PATCH sin cambios	**Aprobado**
gestion de usuarios (admin) — rechazar un registro lo elimina y seudonimiza su rastro	**Aprobado**
gestion de usuarios (admin) — no se puede quitar el ultimo admin: ni bajarle el rol, ni desactivarlo, ni eliminarlo	**Aprobado**
gestion de usuarios (admin) — con un segundo admin, el primero si puede dejar de serlo	**Aprobado**
admin inicial — normaliza el rol previo al RBAC y no pisa un admin existente	**Aprobado**
admin inicial — sin AUTH_USER/AUTH_PASSWORD no hace nada	**Aprobado**

Agrupamiento de alertas · CU-01 / CU-02 · `grouping.test.js`

Riesgo mitigado: Un mismo ataque genera varios incidentes (5712 + 5551): el analista mitiga uno y el otro queda abierto.

Caso de prueba	Resultado
sshd + PAM de la misma IP -> un solo incidente con las dos reglas	**Aprobado**
la misma regla repetida suma alertas pero no se duplica en la lista	**Aprobado**
IPs distintas -> incidentes distintos	**Aprobado**
despues de mitigar, un intento nuevo abre otro incidente	**Aprobado**
fuera de la ventana -> incidente nuevo	**Aprobado**
CU-02: dos archivos maliciosos en el mismo host abierto se agrupan	**Aprobado**
la severidad del incidente es la mas alta de sus alertas	**Aprobado**
se queda con la deteccion del hash mas alta (y la conserva si la alerta nueva no la trae)	**Aprobado**
incidentes anteriores al agrupamiento (ruleIds null) usan su ruleId	**Aprobado**
ventana configurable; 0 la desactiva	**Aprobado**
con el agrupamiento desactivado, cada alerta abre su incidente	**Aprobado**

Métricas de tiempo de respuesta · RNF-05 · `metrics.test.js`

Riesgo mitigado: Cotas de rendimiento calculadas sobre datos inválidos o mezclando incidentes anteriores a la instrumentación.

Caso de prueba	Resultado
calculo — percentil por rango mas cercano: devuelve siempre un valor observado	**Aprobado**
calculo — summarize descarta valores invalidos y maneja el caso vacio	**Aprobado**
calculo — separa por caso de uso y mide los tres tiempos	**Aprobado**
GET /api/v1/metrics — sin token -> 401	**Aprobado**
GET /api/v1/metrics — el webhook registra la ingesta y las acciones registran su duracion	**Aprobado**

Automatización según contexto · CU-01 / CU-02 · `automation.test.js`

Riesgo mitigado: Una acción automática sobre un activo crítico o una IP excluida, una política que se dispara con datos faltantes, o un cambio de configuración sin permisos ni auditoría.

Caso de prueba	Resultado
motor de decision (evaluate) — todas las condiciones se cumplen -> ejecutar en modo activo, simular en modo simulacion	**Aprobado**
motor de decision (evaluate) — modo off: no evalua nada	**Aprobado**
motor de decision (evaluate) — un activo critico nunca se automatiza, aunque una politica lo permita	**Aprobado**
motor de decision (evaluate) — una IP de la lista de exclusion nunca se bloquea (exacta o por CIDR)	**Aprobado**
motor de decision (evaluate) — dato faltante = condicion no cumplida (sin reputacion no se bloquea)	**Aprobado**
motor de decision (evaluate) — respeta el orden: gana la primera politica que aplica	**Aprobado**
motor de decision (evaluate) — solo considera politicas habilitadas y del caso de uso del incidente	**Aprobado**
motor de decision (evaluate) — condiciones de pais, VirusTotal, hash y criticidad	**Aprobado**
motor de decision (evaluate) — ipMatches ignora entradas invalidas	**Aprobado**
hash del malware desde la alerta (CU-02) — lee data.virustotal.positives (formato real de Wazuh), el mock antiguo, o la descripcion	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — CU-02 en modo activo: malware confirmado en un activo de criticidad baja se aisla solo	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — CU-02 en modo simulacion: registra lo que haria y no ejecuta	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — CU-01: decide despues del enriquecimiento (IP con mala reputacion -> bloqueo)	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — CU-01 con reputacion baja: queda para el analista, con el motivo	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — activo critico: nunca automatico	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — si Wazuh rechaza, el incidente queda abierto para el analista y se audita la falla	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — una alerta agrupada vuelve a evaluar: simulacion -> se activa el modo -> la alerta siguiente se contiene	**Aprobado**
de punta a punta (webhook -> decision -> Wazuh) — no decide sobre un incidente que ya no esta abierto, ni sobre uno inexistente	**Aprobado**
configuracion (endpoints) — cualquier rol la ve; solo un admin la cambia	**Aprobado**
configuracion (endpoints) — el admin cambia el modo y la lista de exclusion, y queda auditado	**Aprobado**
configuracion (endpoints) — valida el modo y la lista de exclusion	**Aprobado**
configuracion (endpoints) — el admin habilita una politica y ajusta sus umbrales	**Aprobado**
configuracion (endpoints) — valida las politicas	**Aprobado**
configuracion (endpoints) — inventario de activos: cambia la criticidad o da de alta un agente nuevo	**Aprobado**
configuracion (endpoints) — los valores iniciales no pisan lo que configuro un admin	**Aprobado**

mejoras del 16/10 — lista de exclusion con IPv6 (exacta y rango): antes nunca coincidia y la IP se bloqueaba	**Aprobado**
mejoras del 16/10 — la allowlist rechaza entradas invalidas	**Aprobado**
mejoras del 16/10 — activo fuera del inventario: criticidad alta por defecto, no se automatiza	**Aprobado**
mejoras del 16/10 — cortacircuitos: al llegar al limite pasa sola a simulacion, avisa, y el caso queda para el analista	**Aprobado**
mejoras del 16/10 — el admin ajusta el cortacircuitos y la criticidad por defecto; se valida y se audita antes -> despues	**Aprobado**
mejoras del 16/10 — politicas y activos tambien se auditan con el valor anterior	**Aprobado**
mejoras del 16/10 — precision por politica, con los veredictos de cierre de los analistas	**Aprobado**
metricas — la contencion automatica se mide aparte de la manual	**Aprobado**

Línea de tiempo y auditoría por incidente · Trazabilidad · `timeline.test.js`

Riesgo mitigado: Evidencia del caso perdida al agrupar alertas, o eventos de auditoría que no se pueden atribuir a su incidente.

Caso de prueba	Resultado
dato distintivo de cada alerta — archivo (CU-02), usuario (CU-01) o nada	**Aprobado**
GET /api/v1/incidents/:id/timeline — muestra cada alerta agrupada con su archivo, y los eventos del caso en orden	**Aprobado**
GET /api/v1/incidents/:id/timeline — un incidente anterior al registro de alertas muestra su alerta original	**Aprobado**
GET /api/v1/incidents/:id/timeline — 404 si el incidente no existe, 401 sin token	**Aprobado**
GET /api/v1/audit con filtros — por categoria: solo los tipos de esa categoria, cada uno con su categoria	**Aprobado**
GET /api/v1/audit con filtros — por incidente: solo los eventos de ese caso	**Aprobado**
GET /api/v1/audit con filtros — categoria invalida -> 422; el limite se acota a 200	**Aprobado**
GET /api/v1/audit con filtros — tipos sin categoria conocida quedan como 'other'	**Aprobado**
eventos anteriores al vinculo con el incidente — se vinculan por el numero que mencionan en el texto, una sola vez	**Aprobado**
retencion sobre las alertas del incidente — enmascara la IP de cada alerta vencida	**Aprobado**

Cierre de casos y filtros · Triaje · `closure.test.js`

Riesgo mitigado: Un caso cerrado sin justificación, un falso positivo descartado sin control, o un caso cerrado que sigue absorbiendo alertas.

Caso de prueba	Resultado
validacion del cierre — veredicto obligatorio y del catalogo; etiquetas del catalogo	**Aprobado**
validacion del cierre — benigno y falso positivo exigen una nota de 10+ caracteres	**Aprobado**
validacion del cierre — las etiquetas se unen sin repetir y en el orden del catalogo	**Aprobado**
cerrar y reabrir — el incidente nace con la etiqueta de su caso de uso y suma la de la accion al contener	**Aprobado**
cerrar y reabrir — cierre como verdadero positivo: guarda veredicto, quien, cuando y etiquetas; queda auditado	**Aprobado**
cerrar y reabrir — falso positivo sin nota -> 422; con nota -> cerrado con la nota	**Aprobado**
cerrar y reabrir — no se cierra dos veces, ni algo que no existe; el auditor no puede cerrar	**Aprobado**
cerrar y reabrir — reabrir exige nota, limpia el cierre y deja el veredicto anterior en la auditoria	**Aprobado**
cerrar y reabrir — un caso cerrado no absorbe alertas nuevas: la siguiente abre otro incidente	**Aprobado**
cerrar y reabrir — catalogo de cierre disponible para la app	**Aprobado**
cerrar y reabrir — al eliminar una cuenta, quien cerro el caso tambien se seudonimiza	**Aprobado**
filtros de la lista — abiertos / cerrados	**Aprobado**
filtros de la lista — contenidos / sin contener / por la automatizacion	**Aprobado**
filtros de la lista — veredicto, etiqueta, caso de uso y severidad	**Aprobado**
filtros de la lista — busqueda por numero de incidente, host o IP	**Aprobado**
filtros de la lista — orden por severidad y combinacion de filtros	**Aprobado**
filtros de la lista — valores invalidos -> 422	**Aprobado**
metricas — tiempo hasta el cierre (MTTR)	**Aprobado**

Protección de datos personales · Ley 25.326 · `privacy.test.js`

Riesgo mitigado: Datos personales conservados más de lo necesario, o una baja que borra la trazabilidad de seguridad o no quita el acceso.

Caso de prueba	Resultado
seudonimizacion y enmascarado — el seudonimo es estable, no contiene el username y no es reversible a simple vista	**Aprobado**
seudonimizacion y enmascarado — las IPs se enmascaran a su red, y enmascarar dos veces no cambia nada	**Aprobado**
seudonimizacion y enmascarado — las IPs dentro de un texto de auditoria se enmascaran sin tocar el resto	**Aprobado**
DELETE /api/v1/me (derecho de supresion) — sin token -> 401	**Aprobado**
DELETE /api/v1/me (derecho de supresion) — borra el perfil y seudonimiza la auditoria, sin perder los eventos	**Aprobado**
DELETE /api/v1/me (derecho de supresion) — despues de eliminar la cuenta, el acceso se pierde en el acto	**Aprobado**
DELETE /api/v1/me (derecho de supresion) — el ultimo administrador no puede eliminar su propia cuenta	**Aprobado**
politica de retencion — modo prueba: cuenta lo que cambiaria sin modificar nada	**Aprobado**
politica de retencion — enmascara la IP y descarta el enriquecimiento pasado el plazo; lo reciente queda igual	**Aprobado**
politica de retencion — CU-03: de quien reporto y de los buzones queda solo el dominio	**Aprobado**
politica de retencion — sin parametros usa el plazo por defecto (90 dias)	**Aprobado**

Phishing reportado: ingesta, cuarentena y restauración · CU-03 · `phishing.test.js`

Riesgo mitigado: Un reporte sin el correo original se procesa igual, el mismo correo abre varios incidentes, o la cuarentena informa éxito sin haber movido el correo de ningún buzón.

Caso de prueba	Resultado
parseReport — extrae del original adjunto el Message-ID, remitente, URLs, hash y la IP de envio	**Aprobado**
parseReport — rechaza un reenvio sin el original adjunto (sin Message-ID no se lo puede buscar)	**Aprobado**
parseReport — rechaza un original sin Message-ID	**Aprobado**
ingesta — un reporte crea un incidente CU-03 con el correo, la alerta y la auditoria	**Aprobado**
ingesta — otro reporte del mismo correo se suma al incidente abierto (campana)	**Aprobado**
ingesta — si el incidente del correo ya se cerro, un reporte nuevo abre otro	**Aprobado**
ingesta — VirusTotal confirma el adjunto: severidad alta; un solo motor no alcanza	**Aprobado**
ingesta — respeta el limite de consultas a VirusTotal por correo	**Aprobado**
ingesta — un reporte sin remitente ni asunto igual se ingesta	**Aprobado**
ingesta — si falla el enriquecimiento, el incidente queda igual	**Aprobado**
lectura del buzon de reportes — procesa cada reporte que entrega el proveedor	**Aprobado**
lectura del buzon de reportes — apagada sin configuracion; un error del servidor no corta el intervalo	**Aprobado**
lectura del buzon de reportes — un aviso que llega durante una lectura no se pierde: se lee de nuevo al terminar	**Aprobado**
lectura del buzon de reportes — arranca el aviso por IDLE y el chequeo de respaldo cuando hay configuracion	**Aprobado**
cuarentena y restauracion — pone en cuarentena el correo en todos los buzones donde esta	**Aprobado**
cuarentena y restauracion — restaura, y se puede volver a poner en cuarentena sin perder la primera hora	**Aprobado**
cuarentena y restauracion — acciona solo sobre los correos elegidos (messageIds)	**Aprobado**
cuarentena y restauracion — restaurar parcialmente deja el incidente en cuarentena	**Aprobado**
cuarentena y restauracion — si el correo no esta en ningun buzon, falla y queda auditado	**Aprobado**
cuarentena y restauracion — sin la integracion configurada, la cuarentena y la restauracion fallan con un mensaje claro	**Aprobado**
cuarentena y restauracion — restaurar sin nada en cuarentena falla y queda auditado	**Aprobado**
cuarentena y restauracion — cada accion solo en su caso de uso, y messageIds validado	**Aprobado**
cuarentena y restauracion — el auditor no puede poner en cuarentena ni restaurar	**Aprobado**
integracion con el resto del sistema — filtros por caso de uso y contencion	**Aprobado**
integracion con el resto del sistema — metricas de CU-03: el reporte es la alerta, la cuarentena la contencion	**Aprobado**
integracion con el resto del sistema — backfillUseCases completa los incidentes anteriores a la columna	**Aprobado**

Búsqueda de correos similares de la campaña · CU-03 · `campaign.test.js`

Riesgo mitigado: Correos legítimos tratados como parte de la campaña (por un asunto parecido o un dominio común), candidatos que entran en la cuarentena sin que el analista los elija, o variantes de la misma campaña repartidas en varios incidentes.

Caso de prueba	Resultado
indicadores y puntaje — un indicador fuerte alcanza solo: adjunto, URL, dominio, Reply-To, IP de envio	**Aprobado**
indicadores y puntaje — el remitente solo no alcanza; remitente + asunto parecido si	**Aprobado**
indicadores y puntaje — los dominios propios y comunes no relacionan correos	**Aprobado**
indicadores y puntaje — asunto normalizado: sin RE/FWD ni numeros; los muy cortos no cuentan	**Aprobado**
busqueda en los buzones — al crear el incidente encuentra las variantes de la campana y no los correos legitimos	**Aprobado**
busqueda en los buzones — la cuarentena sin elegir no toca a los candidatos; eligiendolos, si	**Aprobado**
busqueda en los buzones — descartar un candidato: queda auditado y no se puede descartar dos veces	**Aprobado**
busqueda en los buzones — si un usuario reporta un candidato, pasa a confirmado	**Aprobado**
busqueda en los buzones — errores: sin integracion, fuera de CU-03, servidor caido	**Aprobado**
busqueda en los buzones — si falla la busqueda automatica, el incidente se crea igual	**Aprobado**
reportes de otras variantes de la campana — otro Message-ID con el mismo adjunto se suma a la campana abierta, no abre otro caso	**Aprobado**
reportes de otras variantes de la campana — solo un asunto parecido no alcanza para mezclar casos	**Aprobado**

Integración con el servidor de correo (IMAP) · CU-03 · `mailProvider.test.js`

Riesgo mitigado: Una campaña de miles de correos que abre miles de conexiones, reportes procesados dos veces, una sesión que queda abierta tras un error, o un aviso de reporte nuevo que se pierde si la conexión se cae.

Caso de prueba	Resultado
habilitada solo con host, clave del master y buzon de reportes	**Aprobado**
entra a cada buzon con el usuario master por IMAPS	**Aprobado**
cuarentena: mueve el correo a Quarantine en los buzones donde esta	**Aprobado**
restaurar: lo devuelve a la bandeja de entrada	**Aprobado**
reportes: los procesa y los archiva en Processed, o en Invalid si fallan	**Aprobado**
buzon de reportes vacio: no pide mensajes	**Aprobado**
cierra la sesion aunque la operacion falle	**Aprobado**
correos recientes de cada buzon, para la busqueda de similares	**Aprobado**
buzon sin correos desde la fecha: nada que bajar	**Aprobado**
campana de varios correos: una conexion y un solo movimiento por buzon	**Aprobado**
aviso inmediato de reportes (IMAP IDLE) — avisa al conectar y cada vez que llega un correo; cortar cierra la sesion	**Aprobado**
aviso inmediato de reportes (IMAP IDLE) — si la conexion se cae, reconecta	**Aprobado**
aviso inmediato de reportes (IMAP IDLE) — si no puede abrir el buzon, reintenta con espera creciente	**Aprobado**

5. Pruebas de punta a punta contra infraestructura real

Ejecutadas sobre la infraestructura de AWS (manager Wazuh 4.8.2, endpoint víctima con agente Wazuh, orquestador) y la app instalada en un celular Android conectado por Tailscale. Registro completo en `progress.md`.

Fecha	Escenario	Resultado verificado
21/08	Bloqueo de IP vía API de Wazuh	Regla `DROP` creada en `iptables` del endpoint.
21/08	Primer flujo completo desde el celular	Login → incidente → step-up biométrico → bloqueo, por Tailscale. Puerto del orquestador no accesible desde Internet.
26/08	Fuerza bruta SSH real	Regla 5712 disparada; el incidente se crea automáticamente, sin intervención manual.
02/10	CU-02: archivo malicioso (EICAR)	FIM detecta el archivo → VirusTotal (65/75 motores) → alerta 87105 → incidente en el orquestador.
08/10	CU-02: aislar host desde la app	Endpoint sin conexiones nuevas; el agente Wazuh sigue activo (único canal permitido).
08/10	CU-01: bloquear IP desde la app	Conexión TCP del atacante al puerto 22 rechazada tras la acción.
08/10	Enriquecimiento en la app	Tarjeta de inteligencia de amenazas con datos reales de VirusTotal y AbuseIPDB.
13/10	Filtro de ruido	Alerta EICAR nueva llega como incidente; escaneo de integridad posterior no genera incidente. Base depurada de 1.335 a 15 incidentes reales.

13/10	Despliegue continuo	Merge a la rama principal → plan de Terraform sin cambios destructivos → despliegue automático del orquestador vía AWS SSM.
14/10	Campaña de medición, contención manual (CU-01 y CU-02)	12 corridas: contención p50 21,1 s / p95 31,3 s, unas 28 veces menos que los ~10 minutos que insume evaluar una sola alerta en un SOC (Kersten et al., 2023); base de las cotas de los RNF.
16/10	Prueba exploratoria de la app completa en un dispositivo real (realizada por el equipo)	Hallazgos corregidos: el teclado tapaba la nota de cierre, márgenes y etiquetas desparejos, no se podía copiar una IP, el motivo de la automatización se cortaba, y un intento de mitigación rechazado por Wazuh no quedaba en la auditoría (ahora queda, con quién y por qué falló).
17/10	Flujo del analista para contener y cerrar un caso	Recorrido en el dispositivo: de unos 11 toques (biometría en pantalla aparte, volver a buscar el caso para cerrarlo) a 5 (confirmar → huella → éxito → "Cerrar caso" → cerrar, con el veredicto preelegido si se contuvo).
17/10	CU-03: correo reportado → cuarentena desde la app	Incidente creado desde el reporte; el correo sale del INBOX de los dos destinatarios y queda en Cuarentena; restauración y nueva cuarentena verificadas en los buzones.
17/10	CU-03: búsqueda de correos similares	Dos variantes de la campaña (otro asunto y remitente) encontradas como candidatas; el correo legítimo revisado y excluido; la variante descartada quedó en el INBOX.
18/10	Contención automática (CU-01 y CU-02)	6 corridas contenidas por la automatización sin intervención humana: p50 1,51 s / p95 1,96 s.
18/10	CU-03: medición de la contención manual	Contención p50 25,4 s, pero detección p50 6,8 s / p95 11,8 s, fuera de la cota de 5 s (ver defectos).
18/10	CU-03 con IMAP IDLE	Nueva medición tras el cambio: detección p50 0,84 s / p95 1,29 s, las 5 corridas dentro de la cota; contención p50 12,5 s.

6. Defectos detectados por las pruebas

Fecha	Defecto	Cómo se detectó	Resolución
04/08	El manager (agente 000) rechaza acciones de respuesta (error 1703) devolviendo HTTP 200.	Prueba de concepto contra la API	Endpoint víctima dedicado; validación de `total_failed_items` (cubierta en `wazuh.test.js`).
04/08	El bloqueo informaba éxito sin crear la regla de firewall: la IP debía enviarse en `alert.data.srcip`.	Verificación en `iptables`	Cuerpo de la solicitud corregido (cubierto en `wazuh.test.js`).
02/10	Las alertas de malware se rechazaban porque la IP de origen era obligatoria.	Revisión de código con alerta simulada	IP opcional (cubierto en `webhookMalware.test.js`); confirmado horas después con la alerta real.
08/10	Aislar host informaba éxito sin aislar: el script estaba instalado con un nombre distinto al invocado.	Prueba de punta a punta	Instalación corregida y documentada.
13/10	1.320 de 1.334 incidentes eran ruido (actualizaciones del sistema).	Verificación posterior al despliegue	Filtro en el manager y en el orquestador (cubierto en `alertFilter.test.js`).
13/10	Pruebas dependientes de la configuración local (`.env`).	Primera ejecución en CI	Configuración de pruebas autocontenida.

14/10	Un mismo ataque generaba dos incidentes (reglas 5712 y 5551): el analista mitigaba uno y el otro quedaba abierto.	Campaña de medición	Agrupamiento de alertas por agente e IP (cubierto en `grouping.test.js`).
17/10	Las pruebas locales consultaban VirusTotal y AbuseIPDB con las claves reales: la configuración de pruebas las borraba, pero la carga del `.env` las volvía a completar.	Consumo inesperado en el panel de cuota de VirusTotal	Claves vacías (no borradas) en la configuración de pruebas.
17/10	Una prueba dependía del fin de línea del archivo de ejemplo (CRLF en Windows, LF en CI).	Ejecución en CI	Prueba independiente del fin de línea.
18/10	CU-03: detección fuera de la cota de RNF-05 (p50 6,8 s / p95 11,8 s) por el chequeo del buzón de reportes cada 15 s.	Medición de punta a punta	Cambio de diseño a IMAP IDLE con chequeo de respaldo cada 60 s (cubierto en `mailProvider.test.js` y `phishing.test.js`); nueva medición dentro de la cota.
18/10	El agente Wazuh del servidor de correo no podía conectarse al manager (el security group solo aceptaba a la víctima).	Revisión de la arquitectura al dibujar los diagramas	Regla del security group ampliada.

7. Alcance y limitaciones

- **Aplicación móvil.** La cobertura automatizada abarca el backend. La app se valida con las pruebas de punta a punta de la sección 5 en un dispositivo real; incorporar pruebas automatizadas de la capa de servicios de la app queda como trabajo futuro.
- **Frontera con Wazuh.** En las pruebas automatizadas, Wazuh se simula a nivel HTTP. Su comportamiento real (ejecución de la acción en el agente, efecto en el firewall) se valida en las pruebas de punta a punta.
- **Pruebas de punta a punta manuales.** Están documentadas con fecha y evidencia, pero no automatizadas; automatizarlas contra un entorno efímero es una mejora propuesta.